

Object-Oriented Programming & Design Laboratory Work 3

5 points

Instructor: Izbassar Assylzhan

Referenced from: Pakizar Shamoï

Deadline: 12th week, on your practice time. No late submissions will be accepted! Please, read carefully class requirements. Remember about naming conventions. Keep your classes separately.

The lab work requires much more effort & time, rather than practices. For that reason, please, plan your time carefully! Good luck! :)

Key Rules and Guidelines

1. **Prohibition on AI-generated Code.** The use of AI-generated code (e.g., ChatGPT or other similar tools) is strictly prohibited. If AI-generated code is found in your lab, even in a single problem, you will receive 0 points for the entire lab. You are expected to write and develop the code on your own.
2. The 1st two tasks cost **0.25 points**. The 4th and 5th tasks cost **1.5 points**, and 3rd and 6th tasks cost **0.75 points**.

Problem 1

When to use an Interface vs when to use an abstract class. For each “when” provide extended example(s) (with class/interface codes).

Problem 2

Suppose you have an interface `Moveable`. Think of some interface that can extend it and create it. Implement this two interfaces in some classes.

Problem 3

A collection represents a group of objects, known as its elements. Some collections allow duplicate elements and others do not. Some are ordered and others unordered.

Create an Interface `MyCollection` which is **maximum** general (abstract) collection possible.

Problem 4

Create a class called `Employee` whose objects are records for an employee. This class will be a derived class of the class `Person`.

An employee record has an employee's name (inherited from the class `Person`), an annual salary represented as a single value of type `double`, a `hireDate` of type `java.util.Date` (date the employee started work) and a national insuranceNumber, which is a value of type `String`. Inside this class you need to override `toString()` and `equals(Object o)` methods of the `Person` class. Your class should have a reasonable number of constructors and accessor methods.

Then create a class `Manager` extending `Employee`, each manager has a team of employees (`Vector`) and can get a bonus. You need to override `toString()` and `equals(Object o)` methods.

Next, implement `Comparable` interface. (`Employee1 > Employee2` if its salary is more than the salary of `Employee2`, the same for managers, but if their salaries are equal, compare by bonus). In addition, provide 2 comparators – one to sort by name and the other to sort by hire date.

Finally, write a `clone()` method for the `Employee` and `Manager` classes. Use shallow or deep cloning as you want.

Write another class containing a main method to fully test your class definition. Check that your implementations are working fine.

Note: You have to use deep cloning technique when clone the `Manager`.

Problem 5

Write a class `Chocolate` with fields `weight` and `name` (e.g., `Twix`) and method `toString()`. Implement `Comparable` interface (chocolates must be compared by weight).

Implement `Comparable` interface for a `Time` class you created during week 2. Then import this class to your current project.

Next, implement a `Sort` class that will be able to sort anything that can be compared to each other (e.g. anything that is `Comparable`). You have to have a `swap` method that is commonly used in sorting algorithms (this ensures that you will not duplicate this code each time you need to swap elements) and 2 methods for sorting. *USE ANY SORTING ALGORITHMS YOU LIKE*. Bubble Sort and Merge Sort are given just for example.

Code snippet:

```
1 public class Sort {
2     static <E> void swap(E[] array, int i, int j) {
3         /* <E> is the generics here. */
4         // ...
5     }
6
7     static <E extends Comparable<E>> void bubbleSort(E[] array) {
8         /*
9          * <E extends Comparable<E>> means that E must be the type that
10          * extends Comparable interface.
11          */
12         // ...
13     }
```

```
14
15     static <E extends Comparable<E>> void mergeSort(E[] array) {
16         // ...
17     }
18 }
```

Finally, write a `SortTest` class and check that your `Sort` class is capable sorting chocolates, times and employees from previous tasks.

Problem 6

Improve the previous lab's `Person` and `Animal (Pet)` classes by implementing 2 Java build-in interfaces and creating 2 custom interfaces. *You must import that classes, not override them again into this lab work.* Everything in this problem must have sense.